

BACKGROUND

How the web *works.*

The mental map behind every page, API request, login, database row, deploy, and bug you will debug.

DNS

HTTP

browser

API

JSON

database

INTERNET VS WEB

The internet is roads. The web is traffic.

INTERNET

The network.

Cables, routers, wireless signals, data centers, and connected machines around the world.

WEB

One service on it.

Documents and apps accessed with browsers using HTTP and HTTPS.

email, SSH, and games use the internet too. they are not "the web."

URL ANATOMY

A URL is a structured address.

```
https://example.com:443/posts/42?draft=true#comments  
scheme  domain      port path    query    fragment
```

SCHEME

Protocol: usually https.

DOMAIN

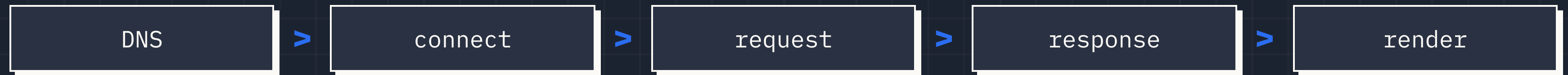
Human-friendly server name.

PATH

Which resource you want.

WHEN YOU PRESS ENTER

One page load is a chain reaction.



The browser hides the complexity. Web developers learn where each part can fail.

DNS

Names for humans. Numbers for machines.

```
google.com -- DNS lookup --> 142.250.80.46
```

DNS is the Domain Name System.

Your browser asks DNS to translate a domain into an IP address, which is where the server can actually be reached.

DNS is the phonebook moment.

TCP + TLS

Connect first. Encrypt next.

TCP

Reliable delivery.

Breaks data into packets, sends them, and makes sure they arrive completely and in order.

TLS

Private delivery.

Encrypts traffic so someone watching the network cannot read the contents.

HTTPS = HTTP over TLS. the padlock means encrypted, not automatically trustworthy.

REQUEST

The browser asks for a resource.

```
GET / HTTP/1.1
Host: google.com
Accept: text/html
User-Agent: Chrome...
```

An HTTP request has a method, path, headers, and sometimes a body.

GET / means: "give me the home resource."

Headers are metadata about the request.

RESPONSE

The server replies with status + body.

```
HTTP/1.1 200 OK
Content-Type: text/html

<!DOCTYPE html>
<html>...</html>
```

An HTTP response has a status code, headers, and usually a body.

The body might be HTML, CSS, JavaScript, JSON, an image, or an error message.

status tells you what happened. body carries the payload.

METHODS

HTTP verbs describe intent.

GET

Retrieve data.

POST

Create something new.

PATCH

Partially update something.

PUT

Replace something completely.

DELETE

Remove something.

RULE

Method + path should read like an action.

STATUS CODES

The first digit is the category.

2XX

Worked

200 OK, 201 Created.

3XX

Redirect/cache

301 moved, 304 not modified.

4XX

Client issue

400 bad request, 401 login, 403 forbidden, 404 not found.

5XX

Server issue. 500 Internal Server Error means the request reached the server, then server code broke or could not complete.

IMPORTANT WEIRDNESS

HTTP is stateless.

Each request is independent. The server does not automatically remember you from the previous request.

COOKIES

Small bits of browser-stored data sent with requests.

SESSIONS

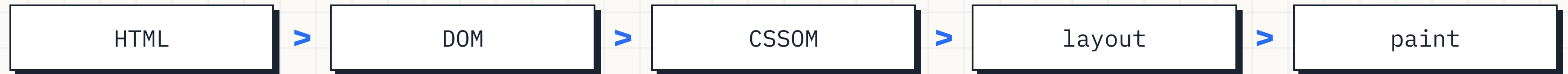
Server-side memory keyed by a cookie.

TOKENS

Signed proof the client sends with requests.

RENDERING

The browser turns files into pixels.



The browser also discovers linked CSS, JavaScript, images, fonts, and data requests, then fetches those too.

JavaScript can change the DOM, which can trigger new layout and paint work.

BROWSER APIS

JavaScript gets browser superpowers.

DOCUMENT

Read and change the page tree.

WINDOW

The browser tab environment.

FETCH

Make HTTP requests from JS.

LOCALSTORAGE

Store simple browser-side data.

DEVTTOOLS

Inspect DOM, console, network, performance.

EVENTS

Clicks, keys, forms, scroll, input.

THREE LANGUAGES

Structure. Style. Behavior.

HTML

What is there?

Headings, paragraphs, images, buttons, forms.

CSS

How does it look?

Color, typography, spacing, layout, animation.

JAVASCRIPT

What does it do?

Clicks, data fetching, state, interaction.

WEBSITE VS WEB APP

Content is different from capability.

WEBSITE

Delivers information.

Usually public, mostly static, often no login or database. Example: article, brochure site, documentation.

WEB APP

Lets users do things.

Interactive, dynamic, user-specific, usually backed by auth and a database. Example: Gmail, Instagram, Google Docs.

this course is mostly web apps.

FRONTEND / BACKEND

Two programs. One user experience.

FRONTEND

Runs in the browser.

- > HTML, CSS, JS, React
- > UI and interaction
- > Displays server data

BACKEND

Runs on a server.

- > Node and Express
- > Business logic and auth
- > Talks to the database

they do not share variables. they talk over HTTP.

API

An API is a communication contract.

GET	/api/posts	all posts
GET	/api/posts/42	one post
POST	/api/posts	create post
PATCH	/api/posts/42	update post
DELETE	/api/posts/42	delete post

In web development, API often means a server that accepts HTTP requests and returns JSON.

A good API route reads like: method + resource + optional ID.

the API is the bridge between UI and data.

JSON

JSON is data as text.

```
{  
  "id": 42,  
  "title": "My First Post",  
  "author": "Ali",  
  "tags": ["react", "javascript"],  
  "published": true  
}
```

JSON looks like a JavaScript object, but it is a text format with stricter rules.

- > Strings use double quotes.
- > Can represent objects, arrays, strings, numbers, booleans, and null.
- > No functions or undefined.

DATABASE

The database is persistent memory.

```
users table:
id | username | email
---+-----+-----
 1 | ali      | ali@example.com
 2 | jess     | jess@example.com
```

```
posts table:
id | title          | user_id
---+-----+-----
 1 | Hello World    | 1
 2 | My Second Post | 1
```

A database stores application data after the server stops.

PostgreSQL is relational: data lives in tables, rows, and columns.

`user_id` is a foreign key: it links each post to the user who wrote it.

PERN

Four tools. One full-stack app.

P

PostgreSQL

Stores data persistently.

E

Express

Handles HTTP requests and defines the API.

R

React

Builds the browser UI.

N

Node.js

Runs JavaScript on the server.

FULL PICTURE

The stack is a chain of responsibilities.

USER'S BROWSER

React

- renders UI
- manages state
- fetches data

|

| HTTP / JSON

v

SERVER

Node + Express

- API routes
- business logic
- authentication

|

| SQL

v

DATABASE

PostgreSQL

- persistent rows

CREATE POST

A click becomes a database row.

1. React calls `axios.post("/api/posts", { title: "Hello", content: "..."} )`
2. Browser sends HTTP POST to Express
3. Express matches `app.post("/api/posts", ...)`
4. Express reads `req.body`
5. Express runs `INSERT INTO posts (...)`
6. PostgreSQL stores the row and returns it
7. Express sends `res.status(201).json(newPost)`
8. Browser receives JSON
9. React updates state
10. React re-renders the UI

this round trip is the course in miniature.

LOCAL VS PRODUCTION

Same ideas. Different address.

LOCALHOST

Your machine.

`http://localhost:3000` means a server running on your own computer.

PRODUCTION

Someone else's machine.

A deployed server on the public internet, usually behind a real domain and HTTPS.

deployment is making localhost public, durable, and secure.

WHEN IT BREAKS

Follow the request.

Where to look

- > Browser console: frontend errors
- > Network tab: request/response truth
- > Server terminal: backend logs
- > Database: persisted data

Ask in order

- > Did the request fire?
- > What URL and method?
- > What status code?
- > What response body?
- > Did React render the result?

UI makes requests. API applies logic. database stores truth. UI re-renders.